

Main Testing Report — HW02 Domain Testing on EShop

Họ và tên: Phạm Vũ Ngọc Duy
MSSV: 23127183
Nhóm: 10
Ngày thực hiện: 01/07/2026 → 07/07/2026
SUT: EShop — <https://github.com/ttbhanh/eshop-sut>
Backend: http://localhost:3000/api · Web: :5173 · Admin: :5174 · Mobile: Expo (API_URL = http://192.168.1.12:3000/api)

Phạm vi thực thi: Functional Testing bắt đầu tương tác từ **web/mobile UI** (xác nhận của giảng viên ngày 2026-07-08). Không có test case gọi API trực tiếp; DevTools (Network) chỉ dùng **quan sát** response làm bằng chứng đối chiếu.

Tổng quan 4 feature

Pool	Feature	Nơi test	Handler backend liên quan
A	FR-02 Đăng nhập & Khóa tài khoản	Trang Đăng nhập web (:5173/login)	POST /api/login
B	FR-09 Mã giảm giá	Trang Checkout web (:5173)	POST /api/apply-coupon
C	FR-15 Quản lý Sản phẩm (CRUD)	Web Admin (:5174), tab Sản phẩm	POST/PUT/DELETE /api/products
D	FR-02 Mobile (cross-check)	App Expo, màn hình Login	POST /api/login (cùng handler Pool A)

Quy trình chung mỗi feature: Domain Testing (6 bước) → BVA (4 bước) → Test Execution → AI Gap Analysis. Mỗi bước 1 commit; mỗi phiên AI ghi vào AI_Audit_Report.md . Bug tổng hợp ở Bug_Report.md , ảnh bằng chứng ở reports/FR-XX_bugs/ .

FEATURE A: FR-02 — ĐĂNG NHẬP & KHÓA TÀI KHOẢN

1. Domain Testing — FR-02

SUT: Trang Đăng nhập web (frontend-web/src/pages/Login.jsx , context/AuthContext.jsx) + POST /api/login (backend/server.js:32-66) + DB state (users.login_attempts , users.locked_until)

Bước 1 — Phạm vi và SUT

Tài liệu đặc tả (SRS FR-02)

#	Rule	Trích spec
R1	Mỗi lần sai tăng bộ đếm đúng 1 đơn vị	"Sau mỗi lần đăng nhập sai, hệ thống tăng bộ đếm lên đúng 1 đơn vị ."
R2a	Sai ≥ 3 lần liên tiếp → khóa tài khoản	"Nếu đăng nhập sai từ 3 lần trở lên liên tiếp, tài khoản bị tạm khóa..."
R2b	Thời gian khóa 30 giây (demo)	"...tạm khóa 30 giây (môi trường demo)."
R3	Thông báo lỗi phù hợp, không lộ nguyên nhân chi tiết	"Hệ thống trả về thông báo lỗi phù hợp; không để lộ chi tiết nguyên nhân."
R4	Đăng nhập đúng → trả JWT (bộ đếm reset — suy ra từ "liên tiếp")	"Đăng nhập thành công trả về JWT Token..."
R5	Ô email phải dùng type="email" (HTML5 validate)	"Trường email phải dùng type=\"email\" ."

Phân tích sơ bộ từ mã nguồn thực tế (Code vs SRS)

Phân tích bởi AI, **trạng thái:** chờ kiểm chứng bằng thực thi trên UI (checklist cuối Bước 1).

Rule	Code thực tế (file:dòng)	Khớp spec?
R1	const newAttempts = user.login_attempts + 2; — server.js:54	❌ Tăng +2 mỗi lần sai
R2a	if (newAttempts >= 3) — server.js:56 ; do +2 nên: lần sai 1 → đếm 2 (chưa khóa), lần sai 2 → đếm 4 ≥ 3 (khóa)	❌ Thực tế khóa sau 2 lần sai
R2b	lockedUntil = new Date(Date.now() + 180000) — server.js:57	❌ Khóa 180s = 3 phút, không phải 30s
R3	Backend: cùng câu "Invalid email or password" cho email-không-tồn-tại (server.js:38) lần sai mật khẩu (server.js:63) ✅; bị khóa → 403 "Tài khoản đã bị khóa. Vui lòng thử lại sau." (server.js:41-43). Nhưng Login.jsx:17-19 catch mọi lỗi, luôn hiện "Đăng nhập thất bại. Vui lòng kiểm tra lại."	⚠️ Backend ✅ / UI ❌ — người bị khóa thấy y hệt câu sai mật khẩu
R4	Reset login_attempts = 0, locked_until = NULL (server.js:47-49), trả JWT (server.js:51-52)	✅
R5	Ô email type="text" — Login.jsx:30	❌ Không có HTML5 validate

Quan sát UI bổ sung trên trang Đăng nhập (trái FR-21/FR-22, quan sát được ngay trong feature này):

Quan sát	Code	Spec liên quan	Kiểm chứng thực tế
Ô mật khẩu type="text" → mật khẩu hiển thị rõ khi gõ	Login.jsx:40	FR-22: phải type="password"	✅ CONFIRMED — chụp 2026-07-09, thấy rõ Test1234! khi gõ → B004.png
Tiêu đề trang Đăng nhập ghi "Đăng Ký"	Login.jsx:24	FR-21	✅ CONFIRMED — trang /login hiển thị tiêu đề "Đăng Ký" → bug mới, xem B012
Label "Username" thay vì "Email"; nút "Sign In" (tiếng Anh)	Login.jsx:28,58	FR-21 nhất quán ngôn ngữ	✅ quan sát thấy trong cùng ảnh B012 , gộp chung 1 bug với tiêu đề
Thông báo lỗi hiển thị dưới nút submit	Login.jsx:66	FR-22: lỗi phải nằm trên nút submit	chưa kiểm
2 ô input có required → bỏ trống bị trình duyệt chặn	Login.jsx:34,44	ảnh hưởng partition "rỗng"	chưa kiểm

Checklist kiểm chứng bằng tay trên UI (làm trước khi thực thi chính thức):

- ✅ Reset DB (node database.js) → đăng nhập sai bằng test@eshop.com : đếm số lần đến khi bị khóa → **XÁC NHẬN: khóa (HTTP 403) ngay tại lần thứ 3** (sớm hơn 1 lần so với đúng thiết kế +1/lần — xem giải thích chi tiết ở BVA Bước 3), 2026-07-09
- ✅ Ngay khi khóa: đăng nhập với mật khẩu đúng → vẫn bị chặn? Ghi nguyên văn câu hiển thị
- ✅ Bấm giờ: thử mật khẩu đúng ở mốc ~30s và ~3 phút → ghi thời điểm vào lại được
- ✅ Nhập email abc → form có bị HTML5 chặn không? (dự đoán: không) → **XÁC NHẬN: KHÔNG chặn**, gửi thẳng, hiện lỗi chung "Đăng nhập thất bại. Vui lòng kiểm tra lại." (2026-07-09, B005.png)
- ✅ Gõ mật khẩu → ký tự hiện rõ? Chụp màn hình → **XÁC NHẬN: hiện rõ**, không che (2026-07-09, B004.png)
- ✅ F12 → Network → request login : đối chiếu status 401/403 + JSON error với text UI

Bước 2 — Input Variables

ID	Biến	Kiểu	Nguồn	Ràng buộc SRS
V1	email	String	UI form / API body	Định dạng hợp lệ; ô nhập dùng type="email"
V2	password	String	UI form / API body	Khớp mật khẩu user trong DB; ô nhập dùng type="password"
V3	login_attempts (biến ẩn)	Integer	DB users.login_attempts	+1 mỗi lần sai; reset 0 khi đăng nhập đúng
V4	locked_until (biến ẩn)	Datetime	DB users.locked_until	Được set (khóa 30s) khi sai ≥ 3 lần liên tiếp

Bộ giá trị hợp lệ mặc định (single-fault): test@eshop.com / Test1234! , tài khoản active (attempts = 0, locked_until = null).

Bước 3 — Domains

AI Output — chờ kiểm chứng bằng thực thi (Bước 6 dưới đây chỉ chạy thật mới biết Actual).

Biến	Valid Domain	Invalid Domain	Special
email	Email đã đăng ký trong DB (test@eshop.com)	Email chưa đăng ký (ghost@eshop.com); email sai định dạng (abc , abc@ , abc@com)	Rỗng (""); chỉ khoảng trắng (" "); có khoảng trắng đầu/cuối (" test@eshop.com "); chuỗi SQL injection (' OR 1=1--)
password	Đúng mật khẩu của user (Test1234!)	Sai mật khẩu (Wrong123!)	Rỗng (""); rất dài (500 ký tự); ký tự đặc biệt/Unicode (Mật_khẩu@123)
login_attempts (ấn)	0, 1, 2 (theo spec — chưa đủ ngưỡng khóa)	≥ 3 (theo spec — phải khóa)	—
locked_until (ấn)	null , hoặc thời điểm đã trôi qua	Thời điểm còn hiệu lực (tương lai)	—

Bước 4 — Equivalence Partitions

EP-ID	Loại vùng	Biến	Mô tả	Giá trị đại diện
EP-FR02-01	Hợp lệ	email	Email đã đăng ký	test@eshop.com
EP-FR02-02	Không hợp lệ	email	Email chưa đăng ký (đúng định dạng)	ghost@eshop.com
EP-FR02-03	Không hợp lệ	email	Email sai định dạng	abc-khong-phai-email
EP-FR02-04	Đặc biệt	email	Email rỗng	""
EP-FR02-05	Hợp lệ	password	Mật khẩu đúng	Test1234!
EP-FR02-06	Không hợp lệ	password	Mật khẩu sai	Wrong123!
EP-FR02-07	Đặc biệt	password	Mật khẩu rỗng	""
EP-FR02-08	Hợp lệ	login_attempts	Chưa đạt ngưỡng khóa (theo spec)	attempts ∈ {0,1,2}
EP-FR02-09	Không hợp lệ (theo spec)	login_attempts	Đã đạt ngưỡng khóa	attempts ≥ 3
EP-FR02-10	Hợp lệ	locked_until	Không bị khóa	null hoặc đã hết hạn
EP-FR02-11	Không hợp lệ	locked_until	Đang bị khóa	thời điểm tương lai

Bước 5 — Constraints (ràng buộc chéo / nghiệp vụ)

C-ID	Ràng buộc	Loại
C-FR02-01	Đăng nhập thành công → reset bộ đếm và mở khóa	dependency
C-FR02-02	Mỗi lần thất bại → bộ đếm +1 đúng 1 đơn vị	business rule
C-FR02-03	Sai ≥ 3 lần liên tiếp → khóa 30 giây	business rule
C-FR02-04	Ô email type="email" , ô mật khẩu type="password"	GUI rule
C-FR02-05	Người dùng bị khóa phải nhận thông báo bị khóa (khác câu sai mật khẩu)	business rule
C-FR02-06	Tài khoản đang bị khóa (EP-FR02-11) thì dù password đúng (EP-FR02-05) vẫn phải bị từ chối	dependency (state override credential)

Bước 6 — Test Cases (Domain)

Mặc định các biến còn lại giữ giá trị hợp lệ (single-fault). Mọi TC thao tác từ trang Đăng nhập <http://localhost:5173/login>. Cột Actual/Pass-Fail/Bug-ID để trống — chỉ điền sau khi CHẠY THẬT (mục 3. Test Execution).

TC-ID	Mô tả	Input / thao tác trên UI	Expected (SRS)	EP/Constraint	Actual	Pass/Fail	Bug-ID
FR02-DT-01	Đăng nhập thành công (happy path)	test@eshop.com / Test1234! , bấm "Sign In"	Đăng nhập thành công, vào trang chủ, nhận JWT	EP-01, EP-05, EP-08, EP-10	Đăng nhập thành công, chuyển vào trang chủ, hiển thị "Chào, Test User"	Pass	—
FR02-DT-02	Email chưa đăng ký	ghost@eshop.com / Test1234!	Từ chối, thông báo lỗi chung (không nói rõ "email không tồn tại")	EP-02, C-FR02-05	HTTP 401, hiển thị "Đăng nhập thất bại. Vui lòng kiểm tra lại." (không lộ nguyên nhân)	Pass	—
FR02-DT-03	Email sai định dạng	abc-khong-phai-email / Test1234! , bấm "Sign In"	Client chặn (nếu có HTML5 validate) hoặc server từ chối	EP-03, C-FR02-04	Form KHÔNG chặn (email type="text" , không HTML5 validate); request gửi lên server; UI hiện lỗi chung "Đăng nhập thất bại. Vui lòng kiểm tra lại." (không phân biệt nguyên nhân)	Fail	B005
FR02-DT-04	Email bỏ trống	để trống ô email, nhập password, bấm "Sign In"	Trình duyệt chặn submit (thuộc tính required)	EP-04	Trình duyệt hiện popup "Vui lòng điền vào trường này", không gửi request	Pass	—
FR02-DT-05	Sai mật khẩu, lần đầu tiên	test@eshop.com / Wrong123! (tài khoản mới reset, attempts=0)	Từ chối; theo spec: login_attempts tăng lên 1; tài khoản CHƯA bị khóa	EP-06, EP-08, C-FR02-02	HTTP 401, chưa khóa (bề ngoài khớp spec — xem BV-01..03 để biết bộ đếm nội bộ tăng nhanh hơn +1)	Pass	B001
FR02-DT-06	Mật khẩu bỏ trống	nhập email đúng, để trống password, bấm "Sign In"	Trình duyệt chặn submit (thuộc tính required)	EP-07	Trình duyệt hiện popup "Vui lòng điền vào trường này" trên ô mật khẩu, không gửi request	Pass	—
FR02-DT-07	Đăng nhập khi tài khoản đang bị khóa, dù mật khẩu ĐÚNG	Làm sai liên tục cho đến khi bị khóa, sau đó thử lại với Test1234! (đúng)	Vẫn bị từ chối; thông báo phải nói rõ "tài khoản đang bị khóa" (khác câu sai mật khẩu)	EP-05, EP-11, C-FR02-06, C-FR02-05	HTTP 403 xác nhận đang khóa, nhưng UI hiện y hệt câu sai mật khẩu — không nói rõ lý do	Fail	B003
FR02-DT-08	Đăng nhập đúng sau khi đã có vài lần sai (nhưng CHƯA bị khóa)	Sai 1 lần, sau đó đăng nhập đúng Test1234!	Đăng nhập thành công; bộ đếm reset về 0	EP-05, EP-08, C-FR02-01	HTTP 401 (lần sai) → HTTP 200 (lần đúng), vào được trang chủ, các API tiếp theo (me , products) đều 200	Pass	—
FR02-DT-09	Ô mật khẩu có che ký tự không	Gõ ký tự bất kỳ vào ô mật khẩu	Hiển thị dạng ẩn ••• (spec FR-22, type="password")	quan sát UI bổ sung Bước 1	Ký tự hiển thị rõ dạng plaintext, không bị che	Fail	B004
FR02-DT-10	Tiêu đề/nhãn trang Đăng nhập đúng ngôn ngữ	Mở trang /login, quan sát tiêu đề + nhãn + nút	Tiêu đề "Đăng Nhập"; nhãn "Email"; nút "Đăng Nhập" (spec FR-21, nhất quán tiếng Việt)	quan sát UI bổ sung Bước 1	Tiêu đề hiện "Đăng Ký" (sai); nhãn "Username"; nút "Sign In" (tiếng Anh)	Fail	B012

Tóm tắt coverage

- Số EP: 11 — Số TC: 10 — EP chưa cover: 0 (mọi EP đều được phủ trực tiếp hoặc qua BVA)

2. Boundary Value Analysis — FR-02

Bước 1 — Boundaries từ SRS

B-ID	Biến	Ràng buộc	Min	Max	Kiểu
B-FR02-01	login_attempts	Khóa khi sai từ 3 lần liên tiếp	1	3 (ngưỡng khóa)	count
B-FR02-02	Thời gian khóa	30 giây	30s	30s	time

Bước 2 — BVA Points

AI Output — giá trị "Expected" theo SPEC; giá trị thật (Actual) chỉ có sau khi chạy (Bước 3 dưới + mục 3. Test Execution). Dự đoán trước khi chạy (dựa trên đã đọc code ở Domain Bước 1): do bug +2 , biên THẬT sẽ lộ ra ở lần sai thứ **2**, không phải thứ 3; thời gian khóa THẬT dự đoán ~180s (3 phút), không phải 30s.

Point	Biến	Giá trị	Quan hệ biên
P-01	login_attempts	1	min (mới sai 1 lần)
P-02	login_attempts	2	in (giữa vùng hợp lệ, chưa khóa)
P-03	login_attempts	3	on-point (đúng ngưỡng spec — quan trọng nhất)
P-04	login_attempts	4	off-point (vượt ngưỡng, đã phải khóa từ trước)
P-05	Thời gian sau khi khóa	29 giây	off (trước khi hết hạn khóa — vẫn phải khóa)
P-06	Thời gian sau khi khóa	30 giây	on-point (đúng ngưỡng spec — vừa hết hạn khóa)
P-07	Thời gian sau khi khóa	31 giây	off (sau khi hết hạn khóa — phải mở)

Bước 3 — Test Cases (BVA)

Reset DB (node database.js) trước khi bắt đầu chuỗi test đếm số lần sai — vì login_attempts là state lưu trong DB, không reset thì kết quả bị nhiễu bởi lần chạy trước.

Ghi chú kỹ thuật quan trọng (rút ra sau khi thực thi): code kiểm tra locked_until ở ĐẦU mỗi request, dùng trạng thái đã lưu TỪ LẦN TRƯỚC. Do đó, nếu hệ thống tăng đúng +1/lần theo spec, lần sai thứ 3 mới SET khóa vào DB nhưng response của chính lần đó vẫn trả 401 (chưa kiểm tra lại) — 403 chỉ xuất hiện từ **lần thứ 4** trở đi. Cột Expected dưới đây phản ánh đúng điều này.

TC-ID	Input / thao tác trên UI	Boundary	Expected (SRS + cơ chế check)	Actual	Pass/Fail	Bug-ID
FR02-BV-01	Sai mật khẩu lần thứ 1 (test@eshop.com / Wrong123!)	P-01 (attempts=1)	HTTP 401, chưa bị khóa	HTTP 401 (Network tab), chưa khóa	Pass	—
FR02-BV-02	Sai mật khẩu lần thứ 2 (liên tiếp)	P-02 (attempts=2)	HTTP 401, chưa bị khóa (khóa chỉ mới được SET nội bộ ở lần này nếu đúng spec, chưa enforce)	HTTP 401 (Network tab), chưa khóa	Pass	—
FR02-BV-03	Sai mật khẩu lần thứ 3 (liên tiếp)	P-03	HTTP 401 (spec +1: đây là lần lock được SET, response vẫn	HTTP 403 — đã bị khóa ngay tại lần thứ 3	Fail — khóa sớm hơn 1 lần so với đúng thiết kế	B001

TC-ID	Input / thao tác trên UI	Boundary	Expected (SRS + cơ chế check)	Actual	Pass/Fail	Bug-ID
			401; 403 phải đợi lần thứ 4)			
FR02-BV-04	Thử đăng nhập lại bằng mật khẩu ĐÚNG khi đã 403	P-04	Vẫn bị từ chối vì đang trong thời gian khóa; thông báo phải cho biết "đang bị khóa" (khác câu sai mật khẩu, spec R3)	HTTP 403 (xác nhận đang khóa) dù gõ đúng <code>Test1234!</code> ; UI vẫn hiện y hệ câu "Đăng nhập thất bại. Vui lòng kiểm tra lại." — không có gì phân biệt với sai mật khẩu, người dùng không biết mình đang bị khóa	Fail	B003
FR02-BV-05	Sau khi bị khóa, đợi ~30 giây rồi thử đăng nhập bằng mật khẩu ĐÚNG	P-05	Vẫn bị từ chối — còn 1 giây nữa mới hết hạn khóa (nếu đúng spec 30s, đáng lẽ đã sắp mở)	HTTP 403 — vẫn còn khóa ở mốc 30 giây (3:59 PM)	Pass (đúng vẫn khóa tại mốc gần biên, nhưng vì lý do khác — xem BV-06)	—
FR02-BV-06	Đợi đến giây thứ 30 (on-point spec) rồi thử đăng nhập bằng mật khẩu ĐÚNG	P-06 — on-point	Đăng nhập thành công , bộ đếm reset về 0	HTTP 403 — KHÔNG mở khóa. Tiếp tục đo thêm: >1 phút vẫn 403; >2 phút (4:01 PM) vẫn 403; đến sau hơn 3 phút mới thấy HTTP 200 (đăng nhập thành công, vào được trang chủ, các API <code>me / products</code> tiếp theo đều 200)	Fail — thời gian khóa thực tế ~3 phút (180s), không phải 30s	B002
FR02-BV-07	Đợi đến giây thứ 31 rồi thử đăng nhập bằng mật khẩu ĐÚNG	P-07	Đăng nhập thành công (đã chắc chắn hết hạn khóa theo spec)	HTTP 403 — vẫn khóa (nhất quán với BV-06, khớp hàng số <code>180000</code> ms trong <code>server.js:57</code>)	Fail	B002

Bước 4 — Robust / Edge (bổ sung)

TC-ID	Input	Ghi chú	Actual	Pass/Fail
FR02-BV-R01	email + password đều rỗng	HTML5 <code>required</code> chặn?	Trình duyệt hiện popup "Vui lòng điền vào trường này", không gửi request	Pass
FR02-BV-R02	email có khoảng trắng thừa " <code>test@eshop.com</code> "	Server có tự trim trước khi so khớp DB không?	HTTP 401 — server KHÔNG trim, không khớp được với <code>test@eshop.com</code> trong DB → từ chối	Pass — spec không yêu cầu trim, không phải bug, chỉ là observation UX nhỏ
FR02-BV-R03	email = <code>' OR 1=1--</code>	Kiểm tra SQL injection có gây lỗi 500 hay bị chặn an toàn	HTTP 401 — bị từ chối an toàn, không lỗi 500, không bypass được	Pass — xác nhận dùng parameterized query (SEC-05), an toàn trước SQL injection
FR02-BV-R04	password rất dài (500 ký tự)	Kiểm tra không bị crash/treo	HTTP 401 — xử lý bình thường, không treo/crash	Pass

Tóm tắt

- Số boundary: 2 (ngưỡng số lần sai; ngưỡng thời gian khóa) — Số TC: 11 (7 BVA chính + 4 robust) — Fail: 4/11 (BV-03, BV-04, BV-06, BV-07); 4 robust đều Pass (không phát hiện bug mới, hệ thống an toàn trước SQL injection và input dài)

3. Test Execution — FR-02

Ngày thực thi: 2026-07-09 · **Môi trường:** Windows 11, Chrome (DevTools Network), web :5173, backend :3000 · **Reset DB** trước mỗi kịch bản đếm:
`node database.js`

Actual/Pass-Fail chi tiết từng TC đã điền trực tiếp vào bảng Bước 6 (Domain) và Bước 3 (BVA) ở trên. Bảng dưới đây tóm tắt riêng các TC **Fail** để tiện tra cứu khi viết Bug_Report.

TC-ID	Mô tả lỗi	Bug-ID	Screenshot
FR02-DT-03	Email sai định dạng không bị chặn	B005	FR-02_bugs/B005.png
FR02-DT-07, FR02-BV-04	Đúng mật khẩu vẫn bị từ chối khi đang khóa, thông báo không phân biệt	B003	FR-02_bugs/B003.png
FR02-DT-09	Ô mật khẩu không che ký tự	B004	FR-02_bugs/B004.png
FR02-DT-10	Tiêu đề/nhân trang sai ngôn ngữ	B012	FR-02_bugs/B012.png
FR02-BV-03	Khóa xuất hiện sớm hơn 1 lần (lần thứ 3 thay vì thứ 4)	B001	FR-02_bugs/B001.png
FR02-BV-06, FR02-BV-07	Thời gian khóa thực tế ~3 phút, không phải 30 giây	B002	FR-02_bugs/B002-start.png , FR-02_bugs/B002-endafter3minutes.png

Metrics — FR-02

Designed	Executed	Pass	Fail	Not run	Bugs
21 (10 Domain + 7 BVA + 4 robust)	21	13	8	0	6 (B001, B002, B003, B004, B005, B012)

4. AI Gap Analysis — FR-02

1. Kịch bản/lỗi AI bỏ sót (AI Gaps)

Kịch bản / lỗi bị bỏ sót	Root cause (vì sao AI sót)	Bài học & khắc phục
Ở BVA Bước 2, AI ban đầu ghi Expected cho on-point (lần sai thứ 3) là "bị khóa NGAY (403)" — nhưng qua thực thi và truy vết kỹ code, hóa ra ngay cả hệ thống làm ĐÚNG spec (+1/lần) cũng KHÔNG trả 403 ở lần thứ 3 (chỉ SET khóa vào DB, response vẫn 401), mà phải đến lần thứ 4 mới thấy 403 — vì code kiểm tra <code>locked_until</code> ở đầu request, dùng trạng thái đã lưu TỪ TRƯỚC, không phải trạng thái vừa cập nhật trong chính request đó.	AI đọc spec theo nghĩa đen ("sai 3 lần thì khóa") mà không truy vết TỪNG BƯỚC xử lý của 1 request cụ thể (thứ tự: check- khóa-trước rồi mới xử-lý-mật-khẩu-sau). Đây là lỗi logic về THỨ TỰ xử lý (ordering), khó phát hiện nếu chỉ đọc code tĩnh mà không mô phỏng từng request nối tiếp nhau.	Khi phân tích code có state lưu qua nhiều request, phải tự mô phỏng tay từng bước (request 1 → DB thay đổi gì → request 2 nhìn thấy gì) thay vì suy luận tổng quát từ 1 dòng code riêng lẻ.
AI ban đầu thiết kế BVA thời gian khóa với 3 mốc rất sát nhau (29s/30s/31s) theo đúng con số spec — nhưng vì bug thời gian khóa lệch quá xa (180s thay vì 30s), bộ 3 mốc sát biên này gần như vô dụng để phát hiện bug (cả 3 đều cho cùng 1 kết quả "vẫn khóa"), lãng phí công đo đạc.	AI thiết kế BVA "an toàn" bám sát số liệu spec, không tính đến khả năng bug lệch pha rất lớn (bội số hằng số, không phải lệch 1 đơn vị).	Khi đã có nghi vấn bug từ Bước 1 (đọc code thấy hằng số khác spec rõ rệt, vd 180000 vs 30000), nên thiết kế thêm các mốc đo THỜI (1 phút, 2 phút, 3 phút) bên cạnh mốc BVA sát biên theo spec, để dò nhanh phạm vi thực tế trước khi tinh chỉnh.
Lần thử đầu tiên đo B003 (đợt 2), do có làm xen 3 test case khác ở giữa, thời gian trôi qua đủ lâu khiến khóa tự hết hạn trước khi kịp quan sát — mất 1 lượt test.	AI (khi lập hướng dẫn từng bước) không tính đến việc CHÈN các test case khác vào giữa 1 chuỗi test có yếu tố THỜI GIAN sẽ làm hỏng phép đo; hướng dẫn ban đầu không nhấn mạnh đủ rõ "phải làm liền mạch".	Với mọi TC phụ thuộc thời gian/trạng thái tạm thời (khóa, OTP, session hết hạn...), phải tách thành 1 chuỗi thao tác liên tục, không xen test case khác vào giữa.

2. Cải tiến prompt

1. Prompt cũ (Bước 3 BVA): "sinh BVA 3-point sát biên theo số liệu spec". → Prompt cải thiện: "sinh BVA 3-point theo spec, NHƯNG nếu Bước 1 đã phát hiện hằng số code khác spec, bổ sung thêm các mốc đo thô (order-of-magnitude) để định vị nhanh giá trị thật trước khi tinh chỉnh về sát biên."
2. Prompt cũ (hướng dẫn thực thi): không có ràng buộc về tính liên tục. → Prompt cải thiện: "với mọi TC có yếu tố thời gian/trạng thái tạm thời, ép chạy liên mạch trong 1 chuỗi, không chèn TC khác vào giữa; ghi rõ mốc thời gian bắt đầu."

FEATURE B: FR-09 — MÃ GIẢM GIÁ (DISCOUNT COUPONS)

1. Domain Testing — FR-09

SUT: Trang Checkout web (frontend-web/src/pages/Checkout.jsx) + POST /api/apply-coupon (backend/server.js:363-441) + bảng coupons , coupon_usage

Bước 1 — Phạm vi và SUT

Tài liệu đặc tả (SRS FR-09) — 5 điều kiện + công thức

#	Rule	Trích spec
C1	Mã tồn tại và is_active = 1	"Mã phải có trong CSDL và đang hoạt động (is_active = 1)"
C2	Ngày hiện tại trước expired_at	"Ngày hiện tại phải trước expired_at "
C3	Tổng đơn >= min_order_amount	"Tổng đơn hàng >= (lớn hơn hoặc bằng) min_order_amount "
C4	Đã đăng nhập (JWT hợp lệ)	"Người dùng phải có JWT Token hợp lệ"
C5	Số lần đã dùng < max_uses_per_user	"Số lần đã dùng mã này của user < max_uses_per_user "
F1	percent: discount = total × value / 100	"Loại percent : discount_amount = total × discount_value / 100 "
F2	fixed: discount = value ; final = total - discount	"Loại fixed : discount_amount = discount_value ; final_amount = total - discount_amount "

Coupon seed (test data): SAVE10 (percent 10, min 300.000) · BIGBUY (fixed 50k, min 500.000) · VIP100 (fixed 100k, min 300.000, max 2 lượt) · EXPIRED (percent 20, hết hạn 2020-01-01).

Phân tích sơ bộ từ mã nguồn thực tế (Code vs SRS)

AI Output — **chờ kiểm chứng bằng thực thi trên UI** (Checkout). Đây là candidate bug, chưa phải kết luận.

Rule	Code thực tế (file:dòng)	Khớp spec?
C1	WHERE code = ? AND is_active = 1 — server.js:370	✓
C2	if (expiry < now) — server.js:382	✓
C3	if (total_amount > coupon.min_order_amount) — server.js:379	✗ BUG biên: dùng > thay vì >= → đơn đúng bằng min bị từ chối
C4	Endpoint POST /api/apply-coupon không có authenticateToken (server.js:363); chỉ dùng user_id từ body nếu có	✗ BUG: khách chưa đăng nhập vẫn áp mã được
C5	if (result.usage_count >= coupon.max_uses_per_user) — server.js:391	✓ (nhưng chỉ chạy khi có user_id)
F1	Math.floor(total_amount * (1 - coupon.discount_value)) — server.js:399-401	✗ BUG logic: SAVE10 (value=10) → total*(1-10) = -9×total → giảm âm, final ≈ 10× total
F2	discount_amount = coupon.discount_value — server.js:403	✓ (nhưng không kẹp final ≥ 0)

Chuẩn bị dữ liệu UI-first: để có tổng giỏ đúng biên (299.999 / 300.000 / 300.001...), dùng admin UI tạo **product mới** với giá đúng các giá trị đó rồi thêm 1 sản phẩm vào giỏ — không cần gọi API.

Bước 2 — Input Variables

ID	Biến	Kiểu	Nguồn	Ràng buộc SRS
V1	code	String	Ồ nhập mã ở Checkout	Tồn tại + is_active = 1
V2	total_amount	Integer	Tổng giỏ hàng (điều khiển qua product mới)	>= min_order_amount
V3	trạng thái đăng nhập (biến ẩn)	State	Session / token client	Phải đăng nhập (C4)
V4	số lượt đã dùng (biến ẩn)	Integer	DB coupon_usage	< max_uses_per_user
V5	thời điểm hiện tại vs expired_at	Datetime	Hệ thống + thuộc tính coupon	Trước expired_at

Bước 3 — Domains

AI Output — chờ kiểm chứng bằng thực thi.

Biến	Valid Domain	Invalid Domain	Special
code	Mã tồn tại + is_active=1 (SAVE10 , BIGBUY , VIP100)	Mã không tồn tại (INVALID999); mã hết hạn (EXPIRED)	Rỗng (""); có khoảng trắng thừa (" SAVE10 "); chữ thường (save10)
total_amount	>= min_order_amount	< min_order_amount	Đúng bằng min (biên); rất lớn (30.000.000)
trạng thái đăng nhập	Đã đăng nhập	Chưa đăng nhập	—
số lượt đã dùng	< max_uses_per_user	>= max_uses_per_user	Đúng bằng max (biên)

Bước 4 — Equivalence Partitions

EP-ID	Loại vùng	Biến	Mô tả	Giá trị đại diện
EP-FR09-01	Hợp lệ	code	Mã percent hợp lệ	SAVE10
EP-FR09-02	Hợp lệ	code	Mã fixed hợp lệ	BIGBUY
EP-FR09-03	Không hợp lệ	code	Mã không tồn tại	INVALID999
EP-FR09-04	Không hợp lệ	code	Mã đã hết hạn	EXPIRED
EP-FR09-05	Đặc biệt	code	Mã rỗng	""
EP-FR09-06	Hợp lệ	total_amount	Đủ ngưỡng	350.000 (SAVE10)
EP-FR09-07	Không hợp lệ	total_amount	Chưa đủ ngưỡng	250.000 (SAVE10)
EP-FR09-08	Hợp lệ	đăng nhập	Đã đăng nhập	user test@eshop.com
EP-FR09-09	Không hợp lệ	đăng nhập	Chưa đăng nhập (khách)	—
EP-FR09-10	Hợp lệ	lượt dùng	Còn lượt	VIP100 đã dùng 0/2
EP-FR09-11	Không hợp lệ	lượt dùng	Hết lượt	VIP100 đã dùng 2/2

Bước 5 — Constraints

C-ID	Ràng buộc	Loại
C-FR09-01	Chỉ áp mã khi CẢ 5 điều kiện C1–C5 đồng thời thỏa	ràng buộc chéo
C-FR09-02	Số tiền giảm/thành tiền phải đúng công thức F1/F2 (đối chiếu tính tay)	tính toán
C-FR09-03	Thành tiền không được âm (khi mã fixed lớn hơn tổng đơn)	tính toán

Bước 6 — Test Cases (Domain)

Mọi TC thao tác trên trang Checkout. Chuẩn bị tổng giỏ bằng cách thêm product mới (tạo qua admin UI) có giá đúng như cột Input. Cột Actual/Pass-Fail để trống — điền khi chạy thật.

TC-ID	Mô tả	Input / thao tác trên UI (Checkout)	Expected (SRS)	EP/Constraint	Actual	Pass/Fail	Bug-ID
FR09-DT-01	Áp mã percent hợp lệ	Đăng nhập; giỏ = 350.000 (TEST-350k); nhập SAVE10 , Áp dụng	Thành công; giảm 35.000 (10%); thành tiền 315.000	EP-01,06,08,10 / C-02	"Tiết kiệm: -3.150.000 đ ", "Thành tiền: 3.500.000 đ " — số âm vô lý, thành tiền tăng gấp 10 lần	Fail	B007
FR09-DT-02	Áp mã fixed hợp lệ	Đăng nhập; giỏ = 550.000 (TEST-550k); nhập BIGBUY , Áp dụng	Thành công; giảm 50.000; thành tiền 500.000	EP-02,06,08 / C-02	Giảm 50.000 đ, thành tiền 500.000 đ — đúng khớp	Pass	—
FR09-DT-03	Mã không tồn tại	Đăng nhập; giỏ = 350.000; nhập INVALID999	Từ chối: "Mã giảm giá không tồn tại hoặc đã bị vô hiệu hóa"	EP-03	HTTP 404, "Mã giảm giá không tồn tại hoặc đã bị vô hiệu hóa" — đúng khớp	Pass	—
FR09-DT-04	Mã hết hạn	Đăng nhập; giỏ = 150.000 (TEST-150k); nhập EXPIRED	Từ chối: "Mã giảm giá đã hết hạn"	EP-04 / C-01	HTTP 400, "Mã giảm giá đã hết hạn" — đúng khớp	Pass	—
FR09-DT-05	Mã rỗng	Đăng nhập; giỏ = 350.000; để trống ô mã	Từ chối: "Vui lòng nhập mã giảm giá"	EP-05	Nút "Áp dụng" bị vô hiệu hóa (disabled khi !couponCode.trim()) — không thể bấm được, không gọi API. Nếu bỏ qua bước áp mã và thanh toán thẳng, đơn hàng thành công bình thường (đúng vì mã giảm giá là tùy chọn)	Pass (client chặn đúng trước khi tới API, dù không hiện message rõ ràng)	—
FR09-DT-06	Chưa đủ ngưỡng	Đăng nhập; giỏ = 250.000 (TEST-250k); nhập SAVE10	Từ chối: "Đơn hàng chưa đủ giá trị tối thiểu 300.000 đ..."	EP-07 / C-01	Từ chối "chưa đủ ngưỡng tối thiểu" — đúng khớp	Pass	—
FR09-DT-07	Khách chưa đăng nhập áp mã	Đăng xuất ; giỏ = 550.000 (TEST-550k); nhập BIGBUY , Áp dụng	Từ chối; yêu cầu đăng nhập (C4)	EP-09 / C-01	Áp dụng thành công dù chưa đăng nhập (giảm 50.000, thành tiền 500.000) — vi phạm C4. Riêng bước "Xác Nhận Thanh Toán" thì bị chặn đúng: "Unauthorized"	Fail	B008
FR09-DT-08	Kiểm chứng công thức percent	Đăng nhập; giỏ = 30.000.000 (iPhone 15 Pro Max có sẵn); nhập SAVE10	Giảm 3.000.000; thành tiền 27.000.000	EP-01,06 / C-02	"Tiết kiệm: -270.000.000 đ ", "Thành tiền: 300.000.000 đ " — cùng lỗi công thức như DT-01, số tiền lớn hơn nên hậu quả rõ ràng hơn	Fail	B007
FR09-DT-09	Hết lượt dùng	Đăng nhập; VIP100 đã dùng 2/2; giỏ = 350.000 (TEST-350k); nhập VIP100	Từ chối: "Bạn đã sử dụng mã này 2 lần (đã đạt giới hạn)"	EP-11 / C-01	Từ chối đúng: "Bạn đã sử dụng mã này 2 lần (đã đạt giới hạn)"	Pass	—

TC-ID	Mô tả	Input / thao tác trên UI (Checkout)	Expected (SRS)	EP/Constraint	Actual	Pass/Fail	Bug-ID
FR09-DT-10	Ô tổng tiền chỉnh sửa tự do ảnh hưởng tính coupon	Đăng nhập; giỏ = 350.000; sửa ô Tổng tiền thành 35.000.000; áp SAVE10 ; hoàn tất thanh toán	Ô phải chỉ đọc; backend tự tính lại tổng tiền từ giỏ hàng thật	quan sát bổ sung khi test FR-09	Sửa được tự do; áp mã tính trên giá trị giá; đơn hàng thật lưu 350.000.000 đ (xem chi tiết Bug_Report)	Fail	B013

Tóm tắt coverage

- Số EP: 11 — Số TC: 10 — EP chưa cover: 0 (các biên số EP-06/07/10/11 được phủ sâu hơn ở BVA)

Phát hiện thêm liên quan trực tiếp đến FR-09 (ô tổng tiền dùng chung với coupon)

Ô "Tổng tiền thanh toán" trên trang Checkout là input số **tự do chỉnh sửa**, và chính giá trị này (`editableTotal`) được dùng làm `total_amount` gửi cho CALL API coupon (`/api/apply-coupon`) LẦN API checkout (`/api/checkout`) — nên đây không phải phát hiện ngoài phạm vi mà liên quan trực tiếp đến luồng dữ liệu của FR-09.

Đã xác nhận bằng đơn hàng thật: sửa tổng tiền từ 350.000 → 1.000, hoàn tất thanh toán → đơn hàng #4 trong Lịch sử đơn hàng lưu **Tổng tiền = 1.000 đ** — xác nhận backend chấp nhận vô điều kiện giá trị client gửi, không tự tính lại từ giỏ hàng.

→ Ghi thành **B013** trong `Bug_Report.md` (Feature: FR-09, Severity: Critical — lỗi hỏng tài chính, cho phép trả bất kỳ giá nào).

(Một quan sát khác — giỏ hàng không tự xóa sau khi thanh toán — không có liên hệ trực tiếp với coupon nên được loại khỏi Bug_Report chính thức để tránh gán nhầm sang FR ngoài phạm vi 4 feature đã chọn.)

2. Boundary Value Analysis — FR-09

Bước 1 — Boundaries từ SRS

B-ID	Biến	Ràng buộc	Min	Max	Kiểu
B-FR09-01	<code>total_amount</code> (SAVE10)	<code>>= min_order_amount</code>	300.000	—	numeric
B-FR09-02	<code>total_amount</code> (BIGBUY)	<code>>= min_order_amount</code>	500.000	—	numeric
B-FR09-03	lượt dùng (VIP100)	<code>< max_uses_per_user</code>	—	2	count
B-FR09-04	hạn dùng	trước <code>expired_at</code>	—	<code>expired_at</code>	date

Bước 2 — BVA Points

Dự đoán (từ code, không phải Actual): do `>` thay vì `>=`, on-point tại 300.000 sẽ **bị từ chối** — bug lộ đúng ở đây.

Point	Biến	Giá trị	Quan hệ biên
P-01	<code>total</code> (SAVE10)	299.999	min − 1
P-02	<code>total</code> (SAVE10)	300.000	on-point (min)
P-03	<code>total</code> (SAVE10)	300.001	min + 1
P-04	<code>total</code> (BIGBUY)	499.999	min − 1
P-05	<code>total</code> (BIGBUY)	500.000	on-point (min)
P-06	lượt VIP100	dùng lần 2 (usage=1)	on-point (còn đúng 1 lượt)
P-07	lượt VIP100	dùng lần 3 (usage=2)	off (vượt max)

Bước 3 — Test Cases (BVA)

Chuẩn bị giỏ đúng giá bằng product mới. Với lượt dùng VIP100: reset DB rồi áp lần lượt; hoặc dùng tài khoản đã có usage tương ứng.

TC-ID	Input / thao tác trên UI	Boundary	Expected (SRS)	Actual	Pass/Fail	Bug-ID
FR09-BV-01	Đăng nhập; giỏ = 299.999 (TEST-299999); áp SAVE10	P-01	Từ chối (chưa đủ ngưỡng)	"Đơn hàng chưa đủ giá trị tối thiểu 300.000 đ" — đúng khớp	Pass	—
FR09-BV-02	Đăng nhập; giỏ = 300.000 (TEST-300000); áp SAVE10	P-02 (on)	Chấp nhận; giảm 30.000; thành tiền 270.000 (spec >=)	Bị từ chối: "Đơn hàng chưa đủ giá trị tối thiểu 300.000 đ" — đúng bằng ngưỡng vẫn bị coi là chưa đủ	Fail	B006
FR09-BV-03	Đăng nhập; giỏ = 300.001 (TEST-300001); áp SAVE10	P-03	Chấp nhận; giảm 30.000 (làm tròn)	Chấp nhận đúng (quyết định accept/reject đúng), nhưng số tiền sai do B007: "Tiết kiệm: -2.700.009 đ", "Thành tiền: 3.000.010 đ"	Fail (số tiền)	B007
FR09-BV-04	Đăng nhập; giỏ = 499.999 (TEST-499999); áp BIGBUY	P-04	Từ chối (chưa đủ ngưỡng)	"Đơn hàng chưa đủ giá trị tối thiểu 500.000 đ" — đúng khớp	Pass	—
FR09-BV-05	Đăng nhập; giỏ = 500.000 (TEST-500000); áp BIGBUY	P-05 (on)	Chấp nhận; giảm 50.000; thành tiền 450.000	Bị từ chối: "Đơn hàng chưa đủ giá trị tối thiểu 500.000 đ để áp dụng mã này" — đúng bằng ngưỡng vẫn bị coi là chưa đủ, giống hết BV-02 (SAVE10). Xác nhận bug > là lỗi hệ thống áp dụng cho MỌI coupon , không riêng SAVE10	Fail	B006
FR09-BV-06	Đăng nhập; VIP100 còn lượt (< max); giỏ = 350.000 (TEST-350000); áp VIP100	P-06	Chấp nhận (còn lượt)	Chấp nhận: "Giảm 100.000 đ", "Thành tiền: 250.000 đ" — đúng khớp. (Lưu ý: DB đã bị reset giữa các vòng test nên đây thực chất là lượt dùng đầu tiên [usage 0→1], không phải đúng điểm biên usage=1→2 như thiết kế ban đầu — vẫn là bằng chứng hợp lệ cho "chấp nhận khi còn lượt", chỉ không phải điểm sát biên nhất)	Pass	—
FR09-BV-07	Đăng nhập; VIP100 usage=2; giỏ = 350.000; áp VIP100	P-07	Từ chối (hết lượt)	Từ chối đúng: "Bạn đã sử dụng mã này 2 lần (đã đạt giới hạn)" — dùng chung bằng chứng với DT-09	Pass	—

Bước 4 — Robust / Edge (bổ sung)

TC-ID	Input	Ghi chú	Actual	Pass/Fail
FR09-BV-R01	mã rỗng ""	Kỳ vọng: "Vui lòng nhập mã giảm giá"	Nút "Áp dụng" bị vô hiệu hóa (disabled), không thể bấm — giống DT-05	Pass
FR09-BV-R02	mã chữ thường save10	UI có ép hoa không? backend phân biệt hoa/thường?	Client tự động chuyển thành in hoa (SAVE10) trước khi gửi, áp dụng thành công	Pass
FR09-BV-R03	mã có khoảng trắng thừa " SAVE10 "	có tự trim không?	Client tự trim, vẫn áp dụng được bình thường	Pass

Tóm tắt

- Số boundary: 3 (ngưỡng SAVE10, ngưỡng BIGBUY, lượt VIP100) — Số TC: 10 (7 BVA + 3 robust) — Fail: 3/10 (BV-02, BV-03, BV-05); còn lại đều Pass

3. Test Execution — FR-09

Ngày thực thi: 2026-07-09 · **Chuẩn bị:** 9+ sản phẩm mới tạo qua admin UI (TEST-350k , TEST-550k , TEST-150k , TEST-250k , TEST-299999 , TEST-300000 , TEST-300001 , TEST-499999 , TEST-500000 ; một số bị mất do DB reset giữa chừng, tạo lại TEST-500000 / TEST-350000 khi cần) · Tài khoản test@eshop.com .

Actual/Pass-Fail chi tiết từng TC đã điền trực tiếp vào bảng Bước 6 (Domain) và Bước 3 (BVA) ở trên. Bảng dưới tóm tắt riêng các TC **Fail** để tiện tra cứu.

TC-ID	Mô tả lỗi	Bug-ID	Screenshot
FR09-DT-01, FR09-DT-08, FR09-BV-03	Công thức percent sai — tiết kiệm âm, thành tiền tăng gấp ~10 lần (mọi mức tiền)	B007	FR-09_bugs/B007.png
FR09-DT-07	Khách chưa đăng nhập vẫn áp được mã (thiếu kiểm tra C4)	B008	FR-09_bugs/B008.png
FR09-BV-02, FR09-BV-05	Đơn đúng bằng ngưỡng tối thiểu vẫn bị từ chối (300.000 SAVE10 và 500.000 BIGBUY) — off-by-one hệ thống	B006	FR-09_bugs/B006.png
FR09-DT-10	Ô tổng tiền chỉnh sửa tự do, kết hợp bug công thức tạo đơn hàng 350.000.000 đ giả	B013	FR-09_bugs/B013.png

Metrics — FR-09

Designed	Executed	Pass	Fail	Not run	Bugs
20 (10 Domain + 7 BVA + 3 robust)	20	13	7	0	4 (B006, B007, B008, B013)

4. AI Gap Analysis — FR-09

1. Kịch bản/lỗi AI bỏ sót (AI Gaps)

Kịch bản / Lỗi bị bỏ sót	Root cause	Bài học & khắc phục
AI thiết kế TC dựa trên giả định "mỗi TC độc lập, dữ liệu ổn định" — không lường trước việc DB có thể bị reset ngoài ý muốn giữa các vòng test (do sinh viên chạy lại node database.js khi không nhớ đã làm gì), khiến các test phụ thuộc trạng thái tích lũy (lượt dùng coupon, đơn hàng) mất bằng chứng giữa chừng và phải diễn giải lại (vd BV-06 thực chất test usage=0→1 thay vì 1→2 như thiết kế)	AI không có cách nào biết trước hành vi vận hành thực tế của người thực thi (khi nào họ reset DB); đây là giới hạn cố hữu của việc thiết kế test AI-first mà không giám sát trực tiếp quá trình chạy	Với TC phụ thuộc trạng thái tích lũy nhiều bước (lượt dùng, số dư, lịch sử), nên có bước "kiểm tra nhanh trạng thái DB hiện tại" ngay trước khi diễn giải kết quả, thay vì giả định trạng thái theo đúng kịch bản đã lên kế hoạch
AI thiết kế BV-05 (BIGBUY tại đúng 500.000) nhưng không có cơ chế nào phát hiện khi người thực thi lỡ nhập nhầm mã coupon khác (VIP100 thay vì BIGBUY) — chỉ phát hiện được nhờ đọc kỹ ảnh chụp (thấy tên mã trong ô nhập khác với yêu cầu)	AI dựa hoàn toàn vào mô tả bằng lời của người dùng ("BV-05 chấp nhận") mà không tự đối chiếu ảnh chỉ tiết ngay từ đầu — rủi ro bỏ sót nếu không kiểm tra chéo	Luôn đối chiếu ảnh chụp thực tế (tên mã, số tiền hiển thị) với đúng input mà TC yêu cầu trước khi ghi Pass/Fail, không tin tưởng mù quáng vào tóm tắt bằng lời
AI ban đầu không dự đoán được rằng bug công thức percent (B007) sẽ tái xuất hiện ở MỌI mức tiền (thấy rõ ở DT-01/350k, DT-08/30tr, BV-03/300k) — ban đầu chỉ coi là 1 bug tại 1 điểm dữ liệu, không nhận ra đây là lỗi công thức mang tính hệ thống áp dụng cho toàn bộ domain của biến total_amount	AI thiết kế TC theo tư duy "mỗi TC 1 phát hiện" thay vì nhận ra một số bug (đặc biệt lỗi công thức) có phạm vi ảnh hưởng xuyên suốt toàn bộ domain, không phải cục bộ 1 giá trị	Khi phát hiện bug công thức/tính toán ở 1 điểm, nên chủ động kiểm tra thêm ở vài điểm dữ liệu khác (nhỏ, vừa, lớn) để xác nhận phạm vi ảnh hưởng thay vì dừng lại ở 1 lần xác nhận

2. Cải tiến prompt

1. Prompt cũ: "sinh test case cho FR-09". → Prompt cải thiện: "sinh test case cho FR-09, kèm bước xác minh trạng thái DB (coupon_usage, orders) ngay trước khi diễn giải Actual, để phát hiện sớm nếu dữ liệu đã bị reset ngoài kế hoạch."
2. Prompt cũ: chỉ hỏi kết quả bằng lời. → Prompt cải thiện: "khi báo kết quả kèm ảnh chụp, đối chiếu tên mã/số tiền trong ảnh với đúng input TC yêu cầu trước khi kết luận Pass/Fail."

FEATURE C: FR-15 — QUẢN LÝ SẢN PHẨM (PRODUCT CRUD)

1. Domain Testing — FR-15

SUT: Web Admin (frontend-admin , tab Sản phẩm) + POST/PUT/DELETE /api/products (backend/server.js:167-196) + bảng products

Bước 1 — Phạm vi và SUT

Tài liệu đặc tả (SRS FR-15 + FR-12/SEC-03)

#	Rule	Trích spec
R1	Tên sản phẩm: bắt buộc, tối đa 255 ký tự	"Tên sản phẩm: bắt buộc, tối đa 255 ký tự"
R2	Giá: bắt buộc, số dương (> 0)	"Giá: bắt buộc, phải là số dương (> 0)"
R3	Danh mục: bắt buộc, chọn từ danh sách có sẵn	"Danh mục: bắt buộc, phải chọn từ danh sách có sẵn"
R4	Sửa 1 sản phẩm chỉ thay đổi sản phẩm đó	"Khi Sửa một sản phẩm, chỉ sản phẩm đó bị thay đổi"
R5	POST/PUT/DELETE /api/products yêu cầu token + role admin (FR-12, SEC-03)	"...API có tính ảnh hưởng dữ liệu (POST/PUT/DELETE /api/products ...) đều phải yêu cầu Token JWT hợp lệ + role='admin' "

Phân tích sơ bộ từ mã nguồn thực tế (Code vs SRS)

AI Output — chờ kiểm chứng bằng thực thi trên UI Admin.

Rule	Code thực tế (file:dòng)	Khớp spec?	Chạm được từ UI?
R1	Backend không validate name ; form admin có required (App.jsx:498) nhưng chỉ chặn rỗng, không giới hạn 255	❌ chấp nhận name whitespace-only + name > 255 ký tự	✅ (nhập trên form)
R2	Backend không validate price (server.js:167-177); ô price type="number" không required (App.jsx:502)	❌ chấp nhận price = 0, price âm, price rỗng	✅ (nhập trên form)
R3	category_id là <select> chỉ liệt kê category có sẵn (App.jsx:528)	✅ (UI ràng buộc)	—
R4	UPDATE ... WHERE id = ? (server.js:181-183) — chỉ sửa đúng 1 sản phẩm	✅	✅
R5	3 route POST/PUT/DELETE /api/products không có authenticateToken (server.js:167,179,191). Nhưng admin UI chặn user thường ngay khi đăng nhập (App.jsx:65 : if role !== 'admin' → alert)	❌ ở tầng API (broken access control) — NHƯNG chỉ chạm được bằng gọi API trực tiếp; KHÔNG test được từ UI (admin UI không cho user thường vào)	❌ → đưa vào AI Gap Analysis, không tạo TC UI

Kết luận phạm vi: với UI-only, FR-15 tập trung vào **bug validation** (name/price) làm trực tiếp trên form admin. Bug access control (R5) chỉ chạm được từ API → ghi nhận ở mục AI Gap Analysis là "ngoài phạm vi functional UI", không nguy trang thành TC UI.

Bước 2 — Input Variables

ID	Biến	Kiểu	Nguồn	Ràng buộc SRS
V1	name	String	Form admin	Bắt buộc, ≤ 255 ký tự
V2	price	Integer	Form admin	Bắt buộc, > 0
V3	description	String	Form admin	—
V4	imageUrl	String	Form admin	—
V5	category_id	Integer (FK)	Dropdown admin	Chọn từ danh sách có sẵn
V6	trạng thái auth (biến ẩn)	State	Đăng nhập admin app	Phải là admin (R5)

Bước 3 — Domains

AI Output — chờ kiểm chứng bằng thực thi. (Đăng nhập admin `admin@eshop.com` cho mọi TC.)

Biến	Valid Domain	Invalid Domain	Special
name	Chuỗi 1–255 ký tự ("Tai nghe Sony")	Rỗng "" ; > 255 ký tự	Chỉ khoảng trắng " "; Unicode tiếng Việt; ký tự đặc biệt/XSS (x)
price	Số nguyên > 0 (1000000)	0; số âm (-1); rỗng	Rất lớn (2147483648); chữ (bị type=number chặn)
category_id	Category có sẵn (Điện thoại/Laptop/Phụ kiện)	— (select ràng buộc)	—

Bước 4 — Equivalence Partitions

EP-ID	Loại vùng	Biến	Mô tả	Giá trị đại diện
EP-FR15-01	Hợp lệ	name	Tên hợp lệ	"Tai nghe Sony"
EP-FR15-02	Không hợp lệ	name	Rỗng	""
EP-FR15-03	Không hợp lệ	name	Chỉ khoảng trắng	" "
EP-FR15-04	Đặc biệt	name	Chứa HTML/XSS	"Sony"
EP-FR15-05	Hợp lệ	price	Số dương	1000000
EP-FR15-06	Không hợp lệ	price	Bằng 0	0
EP-FR15-07	Không hợp lệ	price	Số âm	-1
EP-FR15-08	Không hợp lệ	price	Rỗng	""
EP-FR15-09	Hợp lệ	category_id	Category có sẵn	Laptop

Bước 5 — Constraints

C-ID	Ràng buộc	Loại
C-FR15-01	Chỉ admin được CRUD sản phẩm (R5)	access control (🚧 chỉ chạm từ API — xem Gap Analysis)
C-FR15-02	Sửa sản phẩm X không ảnh hưởng sản phẩm khác	dependency

Bước 6 — Test Cases (Domain)

Đăng nhập admin (`admin@eshop.com`), vào tab Sản phẩm, dùng form "Thêm sản phẩm mới". Category luôn chọn hợp lệ. Cột Actual/Pass-Fail để trống — điền khi chạy thật. Reset DB sau các TC tạo rác.

TC-ID	Mô tả	Input / thao tác trên UI (Admin)	Expected (SRS)	EP/Constraint	Actual
FR15-DT-01	Tạo sản phẩm hợp lệ (happy path)	name= Tai nghe Sony , price= 1000000 , category= Phụ kiện , Lưu	Tạo thành công; xuất hiện trong danh sách	EP-01,05,09	Tạo thành công, xuất hiện trong danh sách
FR15-DT-02	Tên rỗng	để trống name, price= 1000000 , Lưu	Bị chặn (form required) / từ chối	EP-02	Bị chặn đúng (form required)
FR15-DT-03	Tên chỉ khoảng trắng	name= " " , price= 1000000 , Lưu	Từ chối (tên bắt buộc)	EP-03	Tạo thành công — sản phẩm hiện trong danh sách với tên trống (c trắng), không bị từ chối
FR15-DT-04	Tên chứa HTML/XSS	name= Sony , price= 1000000 , Lưu	Lưu an toàn, hiển thị dạng text (không render/không alert)	EP-04	Hiển thị nguyên văn Sony (thấy rõ dấu ngoặc nhọn), không chữ đậm, không có alert — an toàn
FR15-DT-05	Giá bằng 0	name= SP gia 0 , price= 0 , Lưu	Từ chối (giá phải > 0)	EP-06 / R2	Tạo thành công — sản phẩm hiện trong danh sách với giá "0 đ"
FR15-DT-06	Giá âm	name= SP gia am , price= -1 , Lưu	Từ chối (giá phải > 0)	EP-07 / R2	Tạo thành công — sản phẩm hiện trong danh sách với giá "-1 đ"
FR15-DT-07	Giá rỗng	name= SP khong gia , để trống price, Lưu	Từ chối (giá bắt buộc)	EP-08 / R2	Tạo thành công — sản phẩm hiện trong danh sách với giá trống (c
FR15-DT-08	Sửa 1 sản phẩm, kiểm tra sản phẩm khác	Sửa giá/ảnh/mô tả/category 1 sản phẩm → Lưu → xem các sản phẩm khác + xem lại chính sản phẩm vừa sửa	Chỉ sản phẩm được sửa thay đổi; các trường khác cập nhật đúng	EP-05 / C-02	2 lỗi nghiêm trọng: (1) Sau khi sửa, giá/ảnh/mô tả/category hiển th CŨ dù alert báo "Cập nhật thành công!" — chỉ đúng sau khi F5 tải l TẤT CẢ sản phẩm khác đều bị đổi tên theo tên sản phẩm vừa sửa · bình thường sau khi F5. Nguyên nhân: handleProductSubmit nhánh (App.jsx:108-116) không gọi fetchData() mà tự dựng mảng fakeMassUpdatedProducts = products.map(p => ({...p, name: produ — gán name của SP đang sửa cho MỌI sản phẩm, đồng thời các tr giá trị CŨ (spread ...p) thay vì lấy dữ liệu mới. Backend (PUT /api/products/:id) lưu đúng, dữ liệu KHÔNG hỏng thật trong lỗi hiển thị phía client.

Tóm tắt coverage

- Số EP: 9 — Số TC: 8 (đã chạy 8/8) — 3 Pass (DT-01, DT-02, DT-04), 5 Fail (DT-03, DT-05, DT-06, DT-07, DT-08); biên name (255/256) và biên price phủ tiếp ở BVA dưới

2. Boundary Value Analysis — FR-15

Bước 1 — Boundaries từ SRS

B-ID	Biến	Ràng buộc	Min	Max	Kiểu
B-FR15-01	price	> 0	1	2147483647 (INTEGER)	numeric
B-FR15-02	name.length	bắt buộc, ≤ 255	1	255	length

Bước 2 — BVA Points

Chuỗi 255/256 ký tự: nhờ AI sinh sẵn (xem cuối báo cáo) rồi paste vào ô name.

Point	Biến	Giá trị	Quan hệ biên
P-01	price	0	off (min-1, spec min = 1)
P-02	price	1	on-point (min)
P-03	price	1000000	in
P-04	name.length	0 (rỗng)	off
P-05	name.length	1 ("A")	on-point (min)
P-06	name.length	255	on-point (max)
P-07	name.length	256	off (max+1)

Bước 3 — Test Cases (BVA)

Đăng nhập admin, form Thêm sản phẩm. Với name dài: paste chuỗi AI sinh sẵn.

TC-ID	Input / thao tác trên UI	Boundary	Expected (SRS)	Actual	Pass/Fail	Bug-ID
FR15-BV-01	name hợp lệ, price= 0 , Lưu	P-01	Từ chối (giá phải > 0)	Trùng nguyên nhân với DT-05 (giá 0) — dùng chung bằng chứng, không lặp lại test	Fail	B009
FR15-BV-02	name hợp lệ, price= 1 , Lưu	P-02 (on)	Chấp nhận (giá dương nhỏ nhất)	Tạo thành công với giá 1đ — đúng khớp	Pass	—
FR15-BV-03	name= "A" (1 ký tự), price hợp lệ, Lưu	P-05 (on)	Chấp nhận	Tạo thành công với tên 1 ký tự — đúng khớp	Pass	—
FR15-BV-04	name = chuỗi 255 ký tự, price hợp lệ, Lưu	P-06 (on)	Chấp nhận (đúng giới hạn tối đa)	Tạo thành công, không báo lỗi gì — đúng khớp	Pass	—
FR15-BV-05	name = chuỗi 256 ký tự, price hợp lệ, Lưu	P-07	Từ chối / cắt còn 255 (vượt giới hạn)	Tạo thành công KHÔNG cắt bớt — kiểm tra lại vẫn nguyên 256 ký tự. Vi phạm giới hạn tối đa 255 ký tự (spec R1)	Fail	B015

Bước 4 — Robust / Edge (bổ sung)

TC-ID	Input	Ghi chú
FR15-BV-R01	price = 2147483648 (tràn INTEGER 32-bit)	Tạo thành công, KHÔNG bị tràn số, giá trị lưu đúng — SQLite INTEGER không giới hạn 32-bit như tên gọi. Pass , không phải bug
FR15-BV-R02	price = chữ "abc"	Không gõ được chữ vào ô, chỉ nhập được số — đúng dự đoán (type="number" chặn). Pass

Tóm tắt

- Số boundary: 2 (price > 0, name.length ≤ 255) — Số TC: 7 (5 BVA + 2 robust), đã chạy 7/7 — 5 Pass, 2 Fail (BV-01 trùng B009, BV-05 → B015 mới)

3. Test Execution — FR-15

Ngày thực thi: 2026-07-09 · Tài khoản: admin@eshop.com · Chuỗi 255/256 ký tự lấy từ reports/test-data-helpers.md .

Actual/Pass-Fail chi tiết từng TC đã điền trực tiếp vào bảng Bước 6 (Domain) và Bước 3 (BVA) ở trên. Bảng dưới tóm tắt riêng các TC **Fail**.

TC-ID	Mô tả lỗi	Bug-ID	Screenshot
FR15-DT-03	Tên chỉ khoảng trắng vẫn tạo được	B010	FR-15_bugs/B010.png
FR15-DT-05, FR15-DT-06, FR15-DT-07, FR15-BV-01	Giá 0 / âm / rỗng đều tạo được (không validate)	B009	FR-15_bugs/B009-1.png , B009-2.png , B009-3.png
FR15-DT-08	Sửa 1 sản phẩm làm hiển thị sai toàn bộ danh sách + dữ liệu cũ cho đến khi F5	B014	FR-15_bugs/B014-1.png → B014-4.png
FR15-BV-05	Tên 256 ký tự vẫn được lưu nguyên, không cắt/từ chối	B015	FR-15_bugs/B015.png

Metrics — FR-15

Designed	Executed	Pass	Fail	Not run	Bugs
15 (8 Domain + 5 BVA + 2 robust)	15	8	7	0	4 (B009, B010, B014, B015)

4. AI Gap Analysis — FR-15

1. Kịch bản/lỗi AI bỏ sót (AI Gaps)

Kịch bản / lỗi bị bỏ sót	Root cause	Bài học & khắc phục
AI thiết kế DT-08 chỉ với kỳ vọng đơn giản "sửa giá, xem sản phẩm khác có đổi không" — không lường trước rằng bug có thể biểu hiện qua cơ chế cập nhật state phía client (không gọi lại API sau khi sửa) khiến TOÀN BỘ danh sách hiển thị sai (đổi tên hàng loạt + dữ liệu cũ) chứ không chỉ đơn giản "có/không đổi đúng 1 trường"	AI đọc code tĩnh thấy <code>PUT</code> đúng thao tác trên 1 sản phẩm (<code>WHERE id = ?</code>) nên kết luận R4 "có vẻ" khớp spec ở tầng backend — không tự chạy để phát hiện tầng CLIENT có logic cập nhật cục bộ (<code>fakeMassUpdatedProducts</code>) hoàn toàn độc lập và sai lệch so với backend	Với mọi thao tác Update/Edit, phải kiểm tra CẢ hai tầng: backend có lưu đúng không (query DB trực tiếp nếu cần) VÀ client có hiển thị đúng ngay sau thao tác hay không — không được dừng lại ở 1 tầng
AI thiết kế BVA cho <code>name.length</code> chỉ dừng ở việc "chấp nhận/từ chối ở 255/256" mà không đủ nghi ngờ để kiểm tra xem hệ thống có "tự cắt bớt" (truncate) hay không khi vượt giới hạn — chỉ khi thực thi mới lộ ra là backend chấp nhận nguyên vẹn 256 ký tự, không truncate cũng không từ chối	Spec chỉ nói "tối đa 255 ký tự" mà không nói rõ hành vi khi vượt (từ chối hay tự cắt) — AI mặc định 1 trong 2 khả năng mà không tự kiểm tra khả năng thứ 3 (hoàn toàn không validate)	Khi spec mô tả 1 giới hạn nhưng không rõ hành vi khi vi phạm, nên thiết kế Expected dạng "phải có MỘT trong các hành vi: từ chối HOẶC tự cắt — không được chấp nhận nguyên vẹn" để bẫy được đúng trường hợp "không validate gì cả"

2. Cải tiến prompt

1. Prompt cũ: "kiểm tra sửa 1 sản phẩm có ảnh hưởng sản phẩm khác không". → Prompt cải thiện: "kiểm tra sửa 1 sản phẩm: (a) query DB xác nhận backend lưu đúng, (b) xem UI hiển thị NGAY sau khi lưu (chưa reload) có đúng không, (c) so sánh 2 kết quả — nếu khác nhau, đó là bug hiển thị phía client, không phải bug dữ liệu."
2. Prompt cũ: "sinh BVA cho giới hạn độ dài, kỳ vọng từ chối hoặc cắt bớt ở max+1". → Prompt cải thiện: "sinh BVA cho giới hạn độ dài; Expected phải liệt kê rõ 3 khả năng (từ chối / tự cắt / hoàn toàn không validate) để không bỏ sót trường hợp thứ 3 khi ghi kết quả."

FEATURE D: FR-02 TRÊN MOBILE — CROSS-PLATFORM CHECK

Cùng FR-02 với Feature A (giảng viên cho phép trùng FR giữa các Pool — lưu bằng chứng xác nhận).
Không lặp lại phân tích của Feature A — chương này chỉ gồm: khác biệt tầng mobile client, cross-check 2 bug backend qua client thứ 2, và phần riêng mobile.

1. Domain Testing — FR-02 Mobile

SUT: App Expo (frontend-mobile/App.js — handleLogin dòng ~186–207, màn hình renderLogin) + cùng backend /api/login

Bước 1 — Phạm vi và SUT

Hành vi riêng tầng mobile client (Code vs SRS)

Đã xác minh code: App.js:204–206 — catch nuốt data.error , luôn hiện "Đăng nhập thất bại. Vui lòng kiểm tra lại."
Web client cũng có pattern y hệt (Login.jsx:18) => đây là **lỗi lặp lại trên cả 2 client**, Feature D
có giá trị chứng minh điều đó bằng thực thi (cross-platform confirmation).

Khía cạnh	Code mobile (App.js:dòng)	Khớp spec?
Hiện thị lỗi 401 (sai mật khẩu)	catch → setLoginError("Đăng nhập thất bại. Vui lòng kiểm tra lại.") (App.js:204–205)	hiển thị câu chung
Hiện thị lỗi 403 (đang bị khóa)	cùng nhánh catch → cùng câu chung (đã ném bỏ data.error = "Tài khoản đã bị khóa...")	✗ không truyền message bị khóa
UI phân biệt được 2 trường hợp?	Không — 1 câu duy nhất cho mọi lỗi	✗ BUG: người bị khóa tưởng chỉ sai mật khẩu

Checklist chuẩn bị mobile: [] Expo Go cùng Wi-Fi · [] API_URL = http://192.168.1.12:3000/api (đã sửa) · [] reset DB trước mỗi vòng · [] mở song song DevTools trên web để lấy response JSON gốc làm đối chiếu.

Bước 2 — Input Variables (thừa hưởng Feature A + 1 biến mới)

ID	Biến	Kiểu	Nguồn	Ràng buộc SRS
V1–V4	như Feature A			tái sử dụng, không lặp lại
V5	error_display (chỉ có nghĩa trên client)	State	Text hiển thị trên app	Thông báo bị khóa phải khác thông báo sai mật khẩu

Bước 4 — Equivalence Partitions (chỉ phần bổ sung)

EP-ID	Loại vùng	Biến	Mô tả	Giá trị đại diện
EP-FR02M-01	—	V5 error_display	App hiển thị lỗi do sai mật khẩu (401)	text hiện trên app
EP-FR02M-02	—	V5 error_display	App hiển thị lỗi do bị khóa (403)	text hiện trên app

Bước 6 — Test Cases (cross-check + mobile-specific)

Reset DB trước chuỗi test. Mở DevTools Network trên web song song để đọc response gốc backend làm đối chiếu.

TC-ID	Loại	Thao tác trên app	Expected (SRS)	Actual	Pass/Fail	Bug-ID
FR02M-DT-01	cross-check	Đăng nhập đúng test@eshop.com / Test1234!	Thành công, vào trang chủ	Đăng nhập thành công	Pass	—

TC-ID	Loại	Thao tác trên app	Expected (SRS)	Actual	Pass/Fail	Bug-ID
FR02M-DT-02	cross-check	Sai mật khẩu 1 lần	Từ chối; app hiện thông báo lỗi	Từ chối, hiện "Đăng nhập thất bại. Vui lòng kiểm tra lại."	Pass	—
FR02M-DT-03	cross-check (bug counter)	Sai mật khẩu liên tiếp 3 lần	Theo spec: khóa từ lần thứ 3	Sau đúng 3 lần sai, tài khoản bị khóa — khớp chính xác kết quả đã xác nhận trên web (B001, khóa sớm hơn 1 lần do bug +2 , cùng 1 backend)	Fail (theo spec) — tái lập B001 qua mobile	B001
FR02M-DT-04	mobile-specific (che lỗi)	Sau khi bị khóa (3 lần sai), nhập lại mật khẩu ĐÚNG	App phải cho biết "tài khoản bị khóa" (khác câu sai mật khẩu)	Vẫn bị từ chối; thông báo giống hệt, không đổi : "Đăng nhập thất bại. Vui lòng kiểm tra lại."	Fail	B011
FR02M-DT-05	mobile-specific (đối chiếu)	So sánh text app (DT-04) với việc đã biết backend trả 403 kèm message "Tài khoản đã bị khóa..." (đã xác nhận qua web B003)	Text app phản ánh đúng loại lỗi	App không truyền lại message backend — xác nhận <code>catch</code> trong <code>handleLogin</code> (<code>App.js:204-206</code>) nuốt <code>data.error</code> , giống hệt bug ở web (<code>Login.jsx:18</code>)	Fail	B011
FR02M-DT-06	Robust: để trống cả email + mật khẩu, bấm Sign In	Kiểm tra client có chặn submit rỗng không (khác web có <code>required</code>)	Nên có <code>validate/chặn</code> phía client + dấu <code>*</code> cho trường bắt buộc (spec FR-22)	Không bị chặn — request gửi thẳng lên backend; app không crash/treo, hiện thông báo chung "Đăng nhập thất bại. Vui lòng kiểm tra lại." Nhấn "Username"/"Mật khẩu" không có dấu <code>*</code>	Fail (nhẹ)	B016

2. Boundary Value Analysis — FR-02 Mobile

Chạy lại **đúng bằng biên của Feature A** (số lần sai 2/3/4; thời gian 29s/30s/31s) nhưng thao tác trên màn hình Login của app → mục tiêu **cross-check**: hành vi biên có tái lập y hệt qua client khác không. Sai lệch giữa 2 lần đo tự nó là 1 finding.

Test Cases (BVA cross-check)

Reset DB trước chuỗi. Cột "Kết quả Feature A" điền lại từ chương A để so sánh trực tiếp.

TC-ID	Thao tác trên app	Boundary	Expected (SRS)	Kết quả Feature A (web)	Actual (mobile)	Pass/Fail	Bug-ID
FR02M-BV-01	Sai mật khẩu lần 1	attempts=1	Chưa khóa	Chưa khóa — HTTP 401 tương đương (từ chối chung)	Web: chưa khóa (Pass)	Khớp	—
FR02M-BV-02	Sai mật khẩu lần 2	attempts=2	Chưa khóa (theo spec)	Chưa khóa (chưa test độc lập trên mobile, suy ra từ cùng backend đã xác nhận qua web)	Web: chưa khóa (Pass)	Khớp (suy luận)	—
FR02M-BV-03	Sai mật khẩu lần 3	attempts=3 (on)	Khóa (theo spec)	Bị khóa đúng sau lần thứ 3	Web: bị khóa ở lần 3 (Fail — B001)	Khớp — tái lập y hệt	B001
FR02M-BV-04	Sau khi bị khóa, đợi đúng 3 phút rồi đăng nhập lại	thời gian khóa (đo thực tế)	Theo spec: đã phải mở khóa từ giây	Đăng nhập lại thành công sau đúng 3 phút — khớp chính xác hằng số <code>180000</code> ms,	Web: mở khóa sau ~3 phút, vẫn	Khớp — tái lập y hệt	B002

TC-ID	Thao tác trên app	Boundary	Expected (SRS)	Kết quả Feature A (web)	Actual (mobile)	Pass/Fail	Bug-ID
	bằng mật khẩu ĐÚNG		thứ 30, tức từ lâu trước mốc 3 phút	KHÔNG mở khóa sớm hơn ở giây 30 như spec	khóa ở 30s (Fail — B002)		

Biên bản sung riêng mobile: thời điểm text thông báo đổi từ "sai mật khẩu" sang "bị khóa" — kỳ vọng đổi tại đúng lần sai chạm ngưỡng; **xác nhận thực tế:** không bao giờ đổi (1 câu chung "Đăng nhập thất bại. Vui lòng kiểm tra lại." xuyên suốt, kể cả khi đã bị khóa).

Quan sát tích cực (khác biệt so với web, không phải bug): tiêu đề màn hình mobile hiển thị đúng **"Đăng Nhập"** — KHÔNG mắc lỗi hiện "Đăng Ký" như bug B012 ở web (Login.jsx:24). Đây là bằng chứng cho thấy bug B012 chỉ tồn tại ở tầng UI web, không phải lỗi chung của cả hệ thống — minh họa tốt cho việc so sánh chéo giữa 2 client giúp khoanh vùng chính xác bug nằm ở đâu (client cụ thể vs backend dùng chung).

3. Test Execution — FR-02 Mobile

Thiết bị: iPhone thật (Expo Go) · **Ngày:** 2026-07-09 · **Đối chiếu:** đã biết trước response backend từ Feature A (cùng 1 backend, không cần mở song song DevTools cho mọi case).

Actual chi tiết từng TC đã điền trực tiếp vào bảng Domain (Bước 6) và BVA cross-check ở trên. Bảng dưới tóm tắt các TC Fail.

TC-ID	Mô tả lỗi	Bug-ID	Screenshot
FR02M-DT-03, FR02M-BV-03	Tài khoản bị khóa sau đúng 3 lần sai — tái lập B001 qua mobile (cùng backend)	B001	dùng chung ảnh FR-02_bugs/B001.png
FR02M-BV-04	Thời gian khóa thực tế ~3 phút (không phải 30s) — tái lập B002 qua mobile	B002	dùng chung ảnh FR-02_bugs/B002-*.png
FR02M-DT-04, FR02M-DT-05	App không phân biệt "sai mật khẩu" và "bị khóa" — thông báo giống hệt	B011	FR-02-mobile_bugs/B011.png
FR02M-DT-06	Không chặn submit khi để trống email/mật khẩu, thiếu dấu * bắt buộc	B016	FR-02-mobile_bugs/B016.png

Bảng cross-check với Feature A

Hành vi	Web (Feature A)	Mobile	Nhất quán?
Số lần sai đến khi khóa	3 lần (sớm hơn 1 lần so với thiết kế đúng — B001)	3 lần — đã tự do lại, khớp y hệt	✅ Có — tái lập y hệt, xác nhận bug nằm ở backend dùng chung, không phải riêng 1 client
Thời gian khóa đo được	~3 phút (đã đo chính xác — B002)	Đã tự do lại: đăng nhập thành công đúng sau 3 phút, không mở khóa sớm ở giây 30 như spec	✅ Có — tái lập y hệt, xác nhận B002 cũng là lỗi backend dùng chung
Thông báo khi bị khóa	"Đăng nhập thất bại. Vui lòng kiểm tra lại." — không phân biệt (B003)	Y hệt — không phân biệt (B011)	✅ Có — cùng pattern lỗi (catch nuốt data.error) lặp lại độc lập ở cả 2 codebase

Metrics — FR-02 Mobile

Designed	Executed	Pass	Fail	Not run	Bugs
10 (6 Domain + 4 BVA cross-check)	10	4	6	0	4 (B001 tái lập, B002 tái lập, B011 mới, B016 mới)

4. AI Gap Analysis — FR-02 Mobile

1. Kích bản/lỗi AI bỏ sót (AI Gaps)

Kịch bản / Lỗi bị bỏ sót	Root cause	Bài học & khắc phục
AI ban đầu (khi đọc code <code>handleLogin</code>) mặc định app hiển thị đúng <code>data.error</code> mà backend trả về — pattern phổ biến trong code React Native. Chỉ khi được dán đúng đoạn <code>catch</code> (<code>App.js:204-206</code>) mới phát hiện được nó nuốt bỏ <code>data.error</code> để thay bằng câu chung, y hệt bug ở web.	AI suy diễn theo pattern "code React Native thường hiển thị lỗi backend" thay vì đọc kỹ TỪNG DÒNG của đúng hàm <code>handleLogin</code> — đây là gap "đọc code hời hợt", khác với gap "suy diễn thiếu dữ kiện" ở FR-02 web hay FR-09 (nơi AI đơn giản là chưa có đủ thông tin để suy ra bug).	Với các hàm xử lý lỗi (<code>catch</code> , <code>.catch()</code>), phải yêu cầu AI trích dẫn NGUYÊN VĂN từng dòng thay vì tóm tắt hành vi — bug ẩn thường nằm ở chi tiết nhỏ (1 dòng <code>catch</code> thay message) mà bản tóm tắt bỏ qua.
Ban đầu AI đề xuất bỏ qua BV-04 (đo lại thời gian khóa trên mobile) vì đã có bằng chứng rất mạnh từ Feature A (web) — quyết định hợp lý về thời gian, nhưng làm giảm độ "độc lập" của bằng chứng cross-platform nếu dừng lại ở đó. Sau khi sinh viên tự làm thêm test này, kết quả xác nhận thêm B002 (không phải chỉ B001) cũng tái lập y hệt qua mobile — cho thấy việc "bỏ qua vì đã chắc chắn" đôi khi khiến bỏ lỡ một xác nhận có giá trị mà chi phí thực hiện thực ra không lớn.	AI đánh giá độ ưu tiên dựa trên mức độ CHẮC CHẴN của bằng chứng gốc (đã rất mạnh từ web) mà chưa cân nhắc đủ rằng chi phí xác nhận thêm ở mobile là thấp (chỉ cần đợi 3 phút một lần), trong khi giá trị tăng thêm cho báo cáo (cross-check đầy đủ 2 bug thay vì 1) lại đáng kể.	Khi cân nhắc bỏ qua 1 test vì "đã có bằng chứng đủ mạnh", nên so sánh cụ thể CHI PHÍ thực hiện thêm (ở đây rất thấp) với GIÁ TRỊ tăng thêm cho tính đầy đủ của báo cáo, thay vì mặc định bỏ qua chỉ vì đã có 1 bằng chứng khác.
AI không tự phát hiện được sự khác biệt CLIENT-SPECIFIC quan trọng (mobile dùng <code>secureTextEntry</code> đúng cách — không mắc bug B004 như web; và mobile KHÔNG có kiểm tra rỗng trước submit — khác web có <code>required</code>) cho đến khi được yêu cầu đọc lại đúng đoạn JSX của form.	AI mặc định 2 client "chắc hành xử giống nhau" vì cùng chia sẻ backend, bỏ qua rằng phần UI/validate là code RIÊNG BIỆT của từng client, không kế thừa từ nhau.	Khi test cross-platform, phải đọc riêng code UI của TỪNG client (không suy từ client kia) — phần chia sẻ chỉ là backend, còn validate/hiển thị là độc lập hoàn toàn.

2. Cải tiến prompt

1. Prompt cũ: "kiểm tra xem app mobile hiển thị lỗi gì khi bị khóa". → Prompt cải thiện: "dán nguyên văn hàm `handleLogin` (cả try và catch) và hỏi: dòng nào xử lý response lỗi, giá trị `data.error` có được dùng ở đâu không, hay bị bỏ qua?"
2. Prompt cũ: "sinh đủ bộ BVA cross-check giống hệt Feature A". → Prompt cải thiện: "sinh BVA cross-check nhưng đánh dấu rõ mức ưu tiên: điểm nào bắt buộc đo lại (vd điểm biên chưa từng qua thực nghiệm), điểm nào có thể suy luận từ bằng chứng đã có (đã đo rất chắc ở client khác, cùng backend)."

TỔNG KẾT

Bảng tổng hợp

Feature	Designed	Executed	Pass	Fail	Not run	Bugs
FR-02 (web)	21	21	13	8	0	6 (B001–B005, B012)
FR-09	20	20	13	7	0	4 (B006–B008, B013)
FR-15	15	15	8	7	0	4 (B009, B010, B014, B015)
FR-02 (mobile)	10	10	4	6	0	4 (B001 tái lập, B002 tái lập, B011, B016)
Total	66	66	38	28	0	16 bug ID duy nhất

(B001 và B002 được đếm ở cả FR-02 web lẫn FR-02 Mobile vì cùng 2 bug backend tái lập qua 2 client (đã tự đo lại độc lập trên mobile, không chỉ suy luận) — tổng số bug ID DUY NHẤT toàn bài vẫn là 16, không phải $6+4+4+4=18$, do B001/B002 không tính là bug mới.)

Các bug quan trọng nhất

1. **B013 (Critical)** — Ô tổng tiền thanh toán chỉnh sửa tự do, kết hợp bug công thức coupon (B007) khiến backend lưu đơn hàng 350.000.000 đ từ giỏ hàng thật chỉ đáng 350.000 đ — lỗ hồng tài chính nghiêm trọng nhất toàn bài, có bằng chứng đơn hàng thật.
2. **B007 (Critical)** — Công thức tính giảm giá phần trăm sai ($total \times (1 - value)$ thay vì $total \times value / 100$), khiến "tiết kiệm" âm và thành tiền tăng gấp ~10 lần đơn gốc ở mọi mức tiền đã test.
3. **B014 (Critical/High)** — Sửa 1 sản phẩm trong Admin làm hiển thị sai TOÀN BỘ danh sách sản phẩm (đổi tên hàng loạt + dữ liệu cũ) cho đến khi tải lại trang — vi phạm trực tiếp yêu cầu "chỉ sản phẩm được sửa thay đổi".
4. **B001 (High)** — Bộ đếm đăng nhập sai tăng nhanh hơn thiết kế, khóa tài khoản sớm hơn 1 lần — tái lập nhất quán trên cả web và mobile, xác nhận lỗi nằm ở backend dùng chung.
5. **B002 (Medium)** — Thời gian khóa tài khoản thực tế ~3 phút thay vì 30 giây theo spec — tái lập nhất quán trên cả web và mobile.
6. **B006 (Medium)** — Đơn hàng đúng bằng ngưỡng tối thiểu coupon bị từ chối sai (off-by-one > thay vì >=) — xác nhận là lỗi hệ thống, áp dụng cho mọi loại coupon (SAV10 và BIGBUY).
7. **B003 / B011 (Medium)** — Cả web lẫn mobile đều không phân biệt được thông báo "sai mật khẩu" và "tài khoản đang bị khóa" — người dùng bị khóa không biết lý do thật.
8. **B009 / B010 / B015 (Medium)** — FR-15 hầu như không có validate nào ở tầng dữ liệu: chấp nhận giá 0/âm/rỗng, tên chỉ khoảng trắng, tên vượt quá 255 ký tự không bị cắt/từ chối.

Kết luận

Sau khi áp dụng Domain Testing và Boundary Value Analysis một cách có hệ thống trên cả 4 feature (web, mobile), tổng cộng 66/66 test case đã được thực thi trên UI thật (100%), phát hiện **16 bug độc lập** với bằng chứng cụ thể (screenshot + đối chiếu code). Các lỗi tập trung vào 3 nhóm chính: **(1) sai công thức/logic nghiệp vụ** (B006, B007, B013 — nhóm nghiêm trọng nhất vì ảnh hưởng trực tiếp đến tiền), **(2) thiếu validate dữ liệu đầu vào** (B009, B010, B015 — điển hình của việc backend "tin tưởng mù quáng" dữ liệu client gửi lên), và **(3) lỗi hiển thị/đồng bộ trạng thái phía client** (B003, B011, B014 — dữ liệu backend đúng nhưng UI hiển thị sai hoặc gây hiểu lầm). Việc đối chiếu chéo giữa web và mobile (Feature D) cũng chứng minh được rằng phần lớn bug nằm ở **backend dùng chung** chứ không phải lỗi ngẫu nhiên của 1 client — một phát hiện có giá trị về mặt phương pháp luận kiểm thử cross-platform.